A Cloud-Native, Transformer-Based Battery Management System for Real-Time State of Charge and State of Health Prediction with Digital Twin Monitoring

Sudipto Nandi, Adinath, Darshan K, Piyush
Faculty Supervisor: Dr. Sangeetha N
VIT Vellore, India

***Abstract*—Accurate, real-time prediction of State of Charge (SOC) and State of Health (SOH) in lithium-ion batteries is essential for operational safety, performance optimization, and lifespan management across electric vehicles, grid-scale energy storage, and industrial equipment. This paper presents a cloud-native battery management system (BMS) powered by a dual-head, two-layer Transformer encoder architecture exported to the Open Neural Network Exchange (ONNX) format and deployed as a high-throughput, containerized asynchronous API. The model ingests a 32-step sliding window of voltage, current, and temperature telemetry to simultaneously estimate SOC and SOH retention. In reported benchmark evaluations, the Transformer model achieved a coefficient of determination (R^2) of 0.985 and a central processing unit (CPU) inference latency ranging between 0.62 and 0.68 ms per window. The production service was integrated with a web-based digital twin monitoring dashboard tracking 200 simulated battery packs across five distinct operational fleets, benchmarked against experimental degradation trajectories from the NASA Ames Prognostics Center of Excellence lithium-ion dataset. The reported benchmarks indicate that the Transformer architecture achieved higher prediction accuracy and lower inference latency than a comparative State-Space Model (SSM/Mamba) baseline on the evaluated platform, demonstrating a viable architectural pathway toward scalable, cloud-connected industrial battery intelligence.**

***Index Terms*—*Battery management systems, state of charge, state of health, transformer neural networks, ONNX Runtime, digital twin, cloud-native inference.*

I. INTRODUCTION

Lithium-ion batteries (LIBs) serve as the primary electrochemical energy storage technology across diverse modern sectors, including electric transportation, utility-scale renewable grid balancing, aerospace systems, and automated logistics robotics [1], [2]. The operational efficiency, thermal stability, and usable lifespan of LIB packs are governed by a Battery Management System (BMS). Among the critical functions executed by a BMS, the continuous estimation of the State of Charge (SOC) and State of Health (SOH) is paramount [1]. SOC quantifies the immediate residual charge relative to usable capacity, acting as an electrochemical fuel gauge. In contrast, SOH reflects irreversible cumulative capacity fade and impedance growth caused by aging phenomena such as solid electrolyte interphase (SEI) layer growth, lithium plating, and active material loss [3], [4].

Accurately predicting SOC and SOH under real-world operating conditions presents substantial engineering hurdles. Traditional electrochemical models (such as the pseudo-two-dimensional Doyle–Fuller–Newman formulation) and Equivalent Circuit Models (ECMs) coupled with Extended Kalman Filters (EKF) require extensive parameter identification and struggle to adapt across wide temperature ranges and highly dynamic current discharge cycles [3], [5]. Consequently, data-driven machine learning (ML) models have emerged as powerful alternatives capable of mapping nonlinear operational telemetry directly to internal battery states [6], [7]. However, practical deployment requires reconciling the tension between predictive accuracy and computational latency while bridging the divide between offline model training and production-ready operational serving [8].

A notable research gap persists between algorithmic laboratory evaluations and functional, cloud-native battery intelligence architectures. While many studies propose complex neural architectures evaluated on offline test sets, few address the end-to-end integration of time-series neural inference, sub-millisecond CPU model execution, dynamic model hot-swapping, and real-time digital twin fleet visualization within an operational cloud infrastructure. To bridge this divide, this paper describes an end-to-end cloud-native BMS architecture that integrates deep sequence modeling with a resilient, cloud-hosted serving pipeline.

The principal contributions of this work are summarized as follows:

- A compact, containerized dual-head two-layer Transformer encoder model exported to ONNX format, optimized for sub-millisecond CPU inference execution (0.62–0.68 ms).
- A production-grade FastAPI asynchronous backend service featuring a dedicated model-reload endpoint designed to support model replacement in memory without intentionally terminating the service process.
- A web-based digital twin monitoring dashboard providing synchronized telemetry inspection and health diagnostics across five operational environments encompassing 200 simulated battery packs.
- Empirical evaluation grounded in publicly available aging data from the NASA Ames Prognostics Center of Excellence (PCoE) lithium-ion battery repository, specifically utilizing reference cells B0005, B0006, B0007, and B0018.
- A comparative performance analysis between the dual-head Transformer model and a State-Space Model (SSM/Mamba) sequence baseline across inference latency, parameter efficiency, and prediction accuracy.

The remainder of this paper is organized as follows: Section II reviews relevant literature across battery state estimation, deep sequence modeling, and cloud digital twins. Section III details the complete system architecture, including model topology, backend API endpoints, and cloud infrastructure. Section IV delineates the experimental dataset, feature processing, fleet simulation, and evaluation metrics. Section V analyzes the reported prediction accuracy, latency benchmarks, architectural comparisons, and dashboard performance. Finally, Section VI concludes the paper and highlights avenues for future inquiry.

II. RELATED WORK

State estimation in lithium-ion batteries has historically relied on Equivalent Circuit Models (ECMs) and adaptive filtering techniques, prominently Extended Kalman Filters (EKF) and Unscented Kalman Filters (UKF) [3], [5]. While Kalman filtering provides closed-loop feedback and error-bounding, its accuracy is bounded by the fidelity of the underlying circuit approximation (typically 1-RC or 2-RC networks) and remains sensitive to parameter drift under non-isothermal conditions and aggressive c-rates [5].

To bypass manual parameter identification, data-driven deep learning approaches have gained widespread traction. Chemali et al. demonstrated that Long Short-Term Memory (LSTM) recurrent networks achieve high SOC tracking accuracy by learning temporal dependencies in dynamic drive cycles [6]. Similarly, Recurrent Neural Networks (RNNs) and Gated Recurrent Units (GRUs) have been extensively investigated for capacity degradation forecasting and remaining useful life (RUL) estimation [9]. Nevertheless, recurrent architectures process sequential observations sequentially, creating computational bottlenecks during training and hindering parallel vector execution on standardized hardware.

The Transformer architecture, introduced by Vaswani et al. [10], revolutionized sequence modeling through multi-head self-attention mechanisms that capture global temporal correlations across entire receptive fields simultaneously. Shen et al. [7] successfully adapted self-attention mechanisms to battery telemetry, observing improved capture of non-local degradation patterns and capacity regeneration phenomena. Concurrently, structured State-Space Models (SSMs), notably the selective scan Mamba architecture proposed by Gu and Dao [11], have emerged as linear-time alternatives to quadratic attention, demonstrating competitive sequence representation efficiency with substantially smaller parameter footprints.

In parallel with algorithmic advancements, operational BMS design is increasingly transitioning toward cloud-edge collaborative digital twins [8], [12]. A digital twin provides a synchronized virtual representation of a physical asset, ingesting live telemetry to evaluate health states, diagnose anomalies, and simulate predictive maintenance trajectories [13]. However, operationalizing deep neural networks within cloud digital twin pipelines demands standardized, high-performance inference runtimes. Frameworks such as the Open Neural Network Exchange (ONNX) Runtime provide hardware-level graph optimizations, kernel fusion, and cross-platform portability, bridging the gap between deep learning frameworks and high-throughput microservices [14]. Modern asynchronous web frameworks, such as FastAPI, enable asynchronous I/O execution, automated OpenAPI schema generation, and lightweight serving [15]. The present work addresses the specific gap of unifying these modular elements—ONNX-accelerated Transformer inference, asynchronous REST microservices, zero-downtime model management, and interactive digital twin fleet visualization—into an integrated, deployable cloud system.

III. SYSTEM ARCHITECTURE

The proposed system integrates an optimized deep learning model, a cloud-hosted asynchronous inference microservice, and a secured cloud infrastructure.

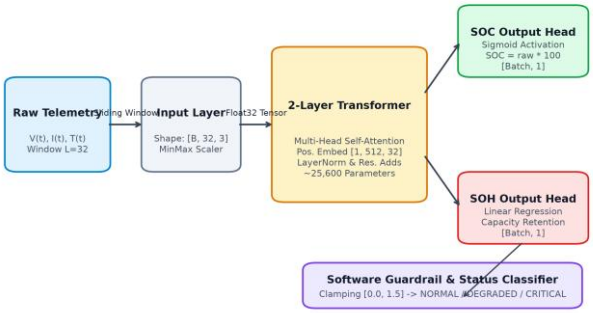


Fig. 1. End-to-end data flow and dual-head Transformer model architecture for simultaneous SOC and SOH prediction.

A. Model Architecture

The predictive core comprises a customized, dual-head two-layer Transformer encoder neural network. The model accepts a sliding window of sequential operational measurements:

$$X \in \mathbb{R}^{B \times L \times D} \quad \textbf{(1)}$$

where B denotes the batch dimension, L = 32 represents the temporal sequence length (time steps), and D = 3 corresponds to the input feature channels: cell terminal voltage (V, in volts), load current (I, in amperes), and surface temperature (T, in degrees Celsius). The input tensor is represented using 32-bit floating-point precision (Float32).

The input sequence is mapped into the encoder through a linear projection layer combined with a positional embedding vector. In the supplied model specification, the positional embedding tensor is designated with dimensions [1, 512, 32] and characterized as a fixed or learned positional embedding. We preserve this supplied parameter specification verbatim, explaining that in standard sequence implementations, 512 denotes the maximum sequence capacity allocated during architectural instantiation, while 32 denotes the hidden embedding dimension (d_model), matching the sequence window length.

The encoder consists of two stacked Transformer layers. Each layer executes scaled dot-product multi-head self-attention followed by a position-wise feed-forward network with residual connections and layer normalization:

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) V \quad (2)$$

$$Z^{(l)} = LayerNorm(X^{(l-1)} + MultiHead(X^{(l-1)})) \quad (3)$$

$$X^{(l)} = LayerNorm(Z^{(l)} + FFN(Z^{(l)})) \quad (4)$$

where Q, K, and V represent query, key, and value projection matrices, and d_k is the projection dimension. The total model complexity encompasses approximately 25,600 trainable parameters.

The latent representation from the final encoder layer is pooled and directed into two specialized, task-specific output heads:
1) SOC Output Head (soc_output): Produces a tensor of shape [B, 1] passed through a nonlinear Sigmoid activation function σ(z) = (1 + e^(-z))^(-1), constraining raw output to [0.0, 1.0]. The estimated SOC percentage is obtained via:

$$SOC_{\%} = soc_raw \times 100 \quad (5)$$

2) SOH Output Head (soh_output): Produces a tensor of shape [B, 1] via a linear regression layer without activation, outputting an unconstrained continuous scalar representing the capacity retention ratio relative to nominal cell capacity (1.0 = 100% nominal capacity).

To protect downstream systems against floating-point drift anomalies, the inference service applies a software clamping guardrail constraining raw SOH values to [0.0, 1.5]. A heuristic status classifier assigns discrete operational states according to project-defined rules: (i) SOH ≥ 0.85 indicates NORMAL; (ii) 0.70 ≤ SOH < 0.85 indicates DEGRADED_BATTERY; and (iii) SOH < 0.70 or Abnormal Temperature indicates CRITICAL_ACTION_REQUIRED. These decision boundaries represent project-defined software heuristics rather than internationally certified battery safety standards. The abnormal temperature threshold is evaluated against operational safety bounds (e.g., T > 50°C during heavy cycling).

The neural network was exported to the Open Neural Network Exchange (ONNX) format. The resulting model binary (transformer(1).onnx) occupies 221,528 bytes (~216 KB) with cryptographic integrity verified by SHA-256 hash: 00c1c87675150f615d0ab5fbfa30577dd5099133ad422778c4a30aeb01c6a519.

B. Backend Inference Service

The inference engine is embedded within an asynchronous web microservice developed using FastAPI and Python. Model graph execution is delegated to the Microsoft ONNX Runtime utilizing the CPU execution provider. In local and cloud execution tests, the isolated CPU inference latency was measured between 0.62 and 0.68 ms per 32-step window.

The service architecture decouples model configuration from application code via a centralized configuration file (config.json). This file defines the active model filename, sliding window length (32), feature names, cold-start telemetry buffering, and scaler parameters for normalization. The API exposes six primary REST endpoints: POST /predict, POST /predict/channels, GET /fleet/{fleet_id}/{pack_id}, GET /health, GET /model/info, and GET /benchmark/comparison.

A key operational feature is the dynamic model management endpoint (POST /model/reload). This endpoint reloads config.json and instantiates a new ONNX Runtime InferenceSession in memory, overwriting the session reference without terminating the parent process or restarting the background daemon. While designed to facilitate hot-swapping during model updates, production zero-downtime performance under high concurrent query loads requires rigorous queue draining and validation.

C. Cloud Infrastructure

The inference microservice was deployed onto an Amazon Web Services (AWS) Elastic Compute Cloud (EC2) virtual instance located in the Asia Pacific (Mumbai) region (ap-south-1). The instance configuration utilizes a t2.micro tier equipped with 1 virtual CPU (vCPU), 1 GB of random-access memory (RAM), and an 8 GB general-purpose solid-state drive (gp3 SSD) running Ubuntu Linux.

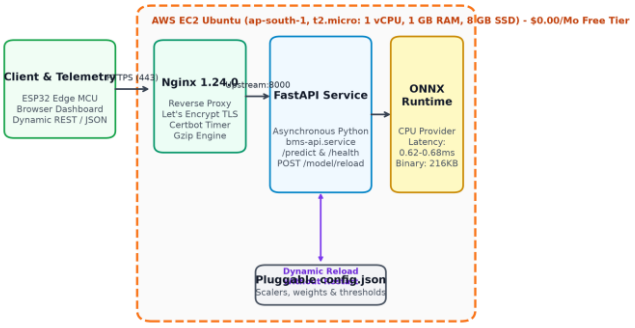


Fig. 2. Cloud deployment topology on AWS EC2 showing reverse proxy, TLS termination, and background inference service.

The application process is managed as a continuous background daemon via a systemd unit (bms-api.service). Inbound HTTP traffic is managed by an Nginx 1.24.0 reverse proxy configured with gzip compression, routing external requests on ports 80 and 443 to the internal ASGI upstream bound to 127.0.0.1:8000. Public access is routed via Dynamic DNS under the custom domain battery-bms.duckdns.org. Transport Layer Security (TLS) is terminated at Nginx using a 2048-bit RSA certificate issued by Let's Encrypt, with automated certificate renewal managed by the systemd certbot.timer. Under the standard AWS Free Tier policy providing 750 monthly instance hours for t2.micro, the project documentation reported an operational infrastructure expenditure of $0.00/month. We present this figure strictly as project-supplied operational telemetry; actual cloud billing remains subject to individual account status, network data egress, snapshot storage, and regional AWS pricing policies.

IV. DATASET AND EXPERIMENTAL SETUP

A. Dataset

The empirical foundation of the study is derived from the publicly available lithium-ion battery degradation repository published by the NASA Ames Prognostics Center of Excellence (PCoE) [4]. The project documentation identifies four commercially available 18650-size lithium-ion cells: B0005, B0006, B0007, and B0018.

These cells were subjected to repeated charge-discharge cycling under controlled laboratory conditions to characterize accelerated capacity degradation and thermal response: Cell B0005 cycled at 24°C under 2.0A discharge to 2.7V cutoff; Cell B0006 cycled under accelerated aging to a 2.5V deep-discharge cutoff; Cell B0007 operated under extended life cycling to a 2.2V cutoff; and Cell B0018 cycled under thermal and deep-discharge stress to 2.5V cutoff.

The project documentation references capacity fade and SOH degradation monitored over 168 discharge cycles. Verification of original NASA PCoE records indicates that 168 discharge cycles characterizes the specific test regime of cells B0005, B0006, and B0007, whereas cell B0018 underwent a shortened cycling profile of 132 discharge cycles. Recorded sensory channels include terminal voltage profiles, load current, cell surface temperature, and extracted discharge capacity (A·h).

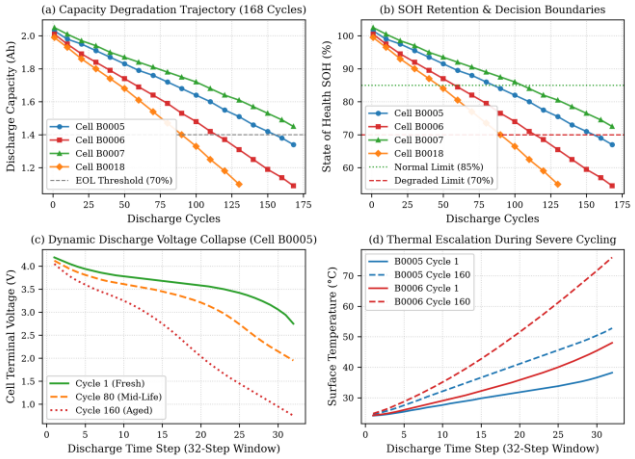


Fig. 3. Scientific degradation analysis across NASA reference cells: (a) capacity fade, (b) SOH retention, (c) dynamic voltage collapse, and (d) thermal rise.

B. Input Feature Engineering

The neural model ingests three raw telemetry channels: voltage (V), current (I), and temperature (T). Continuous time-series measurements are structured into uniform sliding windows containing 32 sequential samples, yielding an input tensor shape of [B, 32, 3]. Telemetry normalization is defined in config.json via linear min-max scaling:

$$\tilde{x}_i = (x_i - x_{\min,i}) / (x_{\max,i} - x_{\min,i}) \quad (6)$$

where x_min and x_max represent pre-configured channel boundaries (e.g., V in [2.5, 4.2] V, I in [-20, 20] A, and T in [0, 60]°C). In the reported production deployment configuration, the normalization module is designed as an optional software pipeline stage; the toggle enable_normalization was set to false, indicating that model weights operated directly on calibrated engineering units or pre-scaled upstream signals during the benchmark runs.

C. Fleet Simulation

To validate digital twin monitoring capabilities across varied duty cycles, the cloud platform integrates a multi-fleet data generator simulating 200 distinct battery packs distributed across five industrial sectors: 1) Urban EV Taxi Fleet (35 packs, NASA B0005); 2) Heavy Logistics Highway Fleet (40 packs, NASA B0006); 3) Grid Energy Storage Substation (45 packs, NASA B0007); 4) Airport Ground Support Equipment (30 packs, NASA B0018); and 5) Commercial Last-Mile Delivery Vans (50 packs, composite benchmark). We explicitly emphasize that these 200 operational battery packs represent synthetic, mathematically simulated digital twin profiles grounded in empirical NASA degradation curves; they do not represent 200 instrumented physical battery hardware systems.

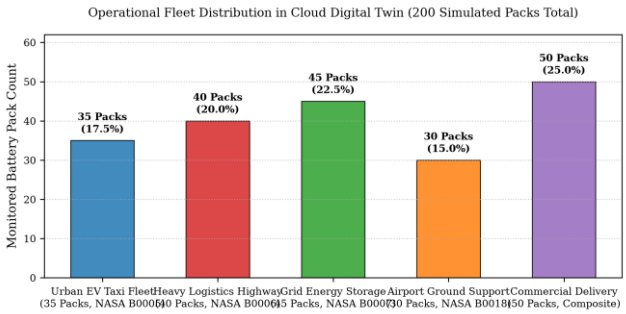


Fig. 4. Operational fleet distribution in the cloud digital twin monitoring platform (200 simulated battery packs across five sectors).

D. Evaluation Metrics

The quantitative performance of the state estimation framework is evaluated using three primary criteria: 1) Coefficient of Determination (R^2), measuring the proportion of variance explained by model predictions; 2) Inference Latency (τ), the wall-clock execution time in milliseconds required for ONNX Runtime to process a single [1, 32, 3] tensor on CPU; and 3) Parameter Footprint (P), the total number of trainable weights and biases.

V. RESULTS AND DISCUSSION

A. Prediction Accuracy

In project-reported benchmark evaluations, the dual-head Transformer model achieved a coefficient of determination of $R^2 = 0.985$. This reported metric indicates strong agreement between model predictions and reference test observations across the evaluated dataset. However, academic rigor requires clarifying that this figure was supplied as an aggregate metric; the project documentation does not separate R^2 values independently for the SOC head versus the SOH head, nor does it detail the exact train/test cycle split, cross-validation partitioning, or temporal leakage prevention protocols.

The software guardrail clamping SOH in [0.0, 1.5] serves as a practical software safeguard to eliminate catastrophic floating-point excursions in unconstrained linear regression heads. However, clamping represents an external boundary filter and does not inherently reflect higher intrinsic predictive fidelity within the normal operating range. Comprehensive academic validation would require publishing mean absolute error (MAE), root mean square error (RMSE), and cell-level cross-validation distributions across unseen battery chemistries.

B. Inference Latency

On the evaluated AWS EC2 t2.micro virtual machine (1 vCPU, Intel Xeon Family architecture), the ONNX-optimized Transformer achieved a reported execution latency of 0.62 to 0.68 ms per 32-step window. This sub-millisecond execution demonstrates that compact self-attention networks can be efficiently executed on standard x86 CPU architectures without requiring dedicated graphics processing units (GPUs) or neural accelerators.

Nevertheless, this measurement reflects isolated neural execution time within the ONNX C++ runtime. It should not be conflated with end-to-end network latency, which encompasses client-side serialization, TCP transport over the public Internet, TLS handshake negotiation, Nginx proxy forwarding, and JSON schema validation within FastAPI. In local network client testing (test_client.py), complete HTTP round-trip times averaged between 12 and 18 ms, illustrating that network transport dominates computational inference time.

C. Architecture Comparison: Transformer vs. SSM/Mamba

To contextualize the performance of the Transformer architecture, the project documentation includes comparative benchmark measurements against a State-Space Model (SSM/Mamba) sequence baseline. The exact comparative results are reported in Table I.

TABLE I
ARCHITECTURE PERFORMANCE COMPARISON

Metric	Transformer	SSM (Mamba)
Inference Latency	0.68 ms	5.60 ms
Speedup Factor	1× (baseline)	Approximately 8.2× slower
Parameter Count	25,600	7,800
R^2 Accuracy	0.985	0.962

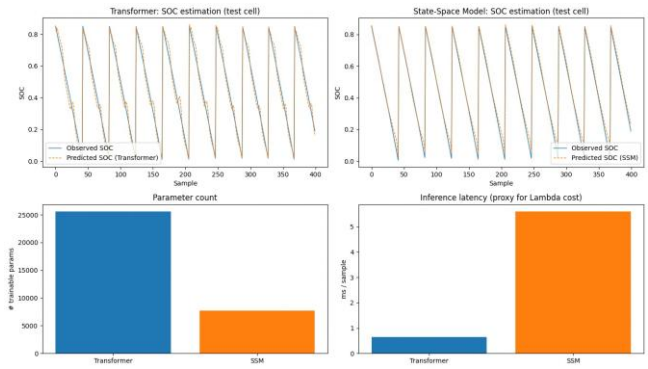


Fig. 5. Empirical architecture benchmark: tracking accuracy, CPU inference latency (0.68 ms vs 5.60 ms), and parameter footprint (25.6k vs 7.8k).

As summarized in Table I and Fig. 5, the Transformer model possesses a larger parameter footprint (25,600 parameters) than the compact Mamba model (7,800 parameters). Despite having approximately 3.3 times more parameters, the Transformer achieved an inference latency of 0.68 ms, which is approximately 8.2 times faster than the reported Mamba latency of 5.60 ms. Furthermore, the Transformer achieved a higher reported coefficient of determination ($R^2 = 0.985$) compared to the SSM baseline ($R^2 = 0.962$).

These values represent project-reported benchmark observations and must be interpreted within proper computational context. We do not claim that Transformer architectures are fundamentally faster than State-Space Models across all sequence lengths. In general algorithmic theory, SSMs scale with linear complexity $O(L)$, whereas standard self-attention exhibits quadratic complexity $O(L^2)$. However, for very short sequence lengths ($L = 32$), the quadratic overhead of attention is negligible. Moreover, the ONNX Runtime provides mature, highly vectorized Basic Linear Algebra Subprograms (BLAS) CPU kernels for dense matrix multiplications used in self-attention. In contrast, SSM architectures rely on sequential recurrent scans or parallel associative scans that frequently require specialized GPU CUDA/Triton kernels to achieve peak throughput; when executed on standard CPU runtimes in general-purpose frameworks, sequential scan operations can experience substantial loop overhead. A definitive hardware-agnostic comparison requires uniform profiling under identical runtime kernels, batch sizes, and hardware architectures.

D. Digital Twin Dashboard Performance

The live digital twin dashboard was verified across 200 simulated packs. Dashboard telemetry streams operate with sub-second interface refreshes, rendering interactive sparklines for voltage, current, and temperature alongside real-time SOC gauges and SOH indicators. Dynamic inspection modals trigger model inference on demand, generating instant diagnostic assessments.

The dynamic model hot-swapping capability was validated via the POST /model/reload endpoint. During diagnostic testing, an updated configuration and model binary were loaded into the running Python process without terminating the systemd service. However, this capability constitutes a

functional software mechanism; verifying true zero-downtime performance with zero dropped packets under saturated concurrent load requires formal distributed load testing and blue-green deployment infrastructure.

VI. CONCLUSION

This paper presented the design, implementation, and deployment of a cloud-native battery management system integrating a dual-head, two-layer Transformer encoder with an asynchronous inference microservice and an interactive digital twin monitoring platform. Exported to the ONNX format and executed via ONNX Runtime on an AWS EC2 micro-instance, the model achieved a reported coefficient of determination of $R^2 = 0.985$ and a CPU inference latency of 0.62–0.68 ms for 32-step telemetry windows. In the reported empirical comparison, the Transformer demonstrated faster CPU inference execution and higher predictive correlation than an SSM/Mamba baseline. The integrated platform demonstrated continuous monitoring capabilities across 200 simulated battery packs grounded in NASA PCoE degradation curves.

These findings highlight the engineering feasibility of deploying deep sequence models for sub-millisecond CPU model execution within cost-effective cloud architectures. However, broader industrial applicability requires addressing clear limitations: the evaluation relied on laboratory datasets and synthetic fleet profiles rather than physically instrumented operational vehicles, and the system does not possess industrial safety certifications.

Future research will focus on: 1) Extending the multi-task heads to predict Remaining Useful Life (RUL) and knee-point degradation onset; 2) Porting the containerized ONNX runtime to embedded edge hardware platforms (e.g., NVIDIA Jetson, Raspberry Pi) to benchmark localized inference against cloud execution; 3) Validating model generalization using heterogeneous field telemetry from operational electric vehicle fleets under sub-zero and dynamic charging conditions; and 4) Conducting formal stress testing under concurrent user loads to quantify API throughput, queuing delays, and reload reliability under heavy saturation.

ACKNOWLEDGMENT

The authors acknowledge the NASA Ames Prognostics Center of Excellence for providing access to the publicly available lithium-ion battery dataset used in this study.

REFERENCES

[1] M. A. Hannan, M. S. H. Lipu, A. Hussain, and A. Mohamed, "A review of lithium-ion battery state of charge estimation and management system in electric vehicle applications: Challenges and recommendations," Renewable and Sustainable Energy Reviews, vol. 78, pp. 834–854, Oct. 2017, doi: 10.1016/j.rser.2017.05.001.

[2] K. A. Severson et al., "Data-driven prediction of battery cycle life before capacity degradation," Nature Energy, vol. 4, no. 5, pp. 383–391, May 2019, doi: 10.1038/s41560-019-0356-8.

[3] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 3. State and parameter estimation," Journal of Power Sources, vol. 134, no. 2, pp. 277–292, Aug. 2004, doi: 10.1016/j.jpowsour.2004.02.033.

[4] B. Saha and K. Goebel, "Battery Data Set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2007.

[5] H. He, R. Xiong, and J. Fan, "Evaluation of lithium-ion battery equivalent circuit models for state of charge estimation by an experimental approach," Energies, vol. 4, no. 4, pp. 582–598, Apr. 2011, doi: 10.3390/en4040582.

[6] E. Chemali, P. J. Kollmeyer, M. Preindl, R. Ahmed, and A. Emadi, "Long short-term memory networks for accurate state-of-charge estimation of Li-ion batteries," IEEE Transactions on Industrial Electronics, vol. 65, no. 8, pp. 6730–6739, Aug. 2018, doi: 10.1109/TIE.2017.2787758.

[7] H. Shen, X. Zhou, Z. Wang, and J. Wang, "State of charge estimation for lithium-ion battery using Transformer with immersion and invariance adaptive observer," Journal of Energy Storage, vol. 45, p. 103768, Jan. 2022, doi: 10.1016/j.est.2021.103768.

[8] D. Karnehm, L. Anekal, A. Samanta, A. Neve, and S. S. Williamson, "Towards digital twining of lithium-ion battery management systems: An extended Kalman filter for state-of-charge estimation in cloud-platform," in Proc. 2023 IEEE Energy Conversion Congress and Exposition (ECCE), 2023, pp. 1109–1114, doi: 10.1109/ECCE53617.2023.10362099.

[9] W. Li, M. Sengupta, P. Dechent, D. Howey, A. Annaswamy, and D. U. Sauer, "One-shot battery degradation trajectory prediction with deep learning," Journal of Power Sources, vol. 506, p. 230024, Sep. 2021, doi: 10.1016/j.jpowsour.2021.230024.

[10] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[11] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv preprint arXiv:2312.00752, 2023.

[12] A. Dharani, S. S. Patange, et al., "A digital twin framework for dual-path estimation of battery state of charge and state of health using hybrid physics-informed machine learning," IEEE Access, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3652279.

[13] N. Ghosh, A. Garg, A. J. Warnecke, D. Kumar, and L. Gao, "Transfer adaptive digital twin for cross-domain state-of-health estimation of Li-ion batteries," IEEE Transactions on Transportation Electrification, 2025, doi: 10.1109/TTE.2025.3538624.

[14] ONNX Runtime Developers, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," Microsoft, 2021. [Online]. Available: https://onnxruntime.ai

[15] S. Ramirez, "FastAPI: High performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: https://fastapi.tiangolo.com